

Technical University of Munich

School of Computation, Information and Technology
-- Informatics --

Bachelor's Thesis in Informatics

Self-Service Management of Time-Limited Access to Heterogeneous Resources

Self-Service-Verwaltung zeitlich begrenzter Zugriffe auf heterogene Ressourcen

Author:	Benjamin Schmitz
Examiner:	Prof. Dr. Stephan Krusche
Start Date:	01.10.2026
Submission Date:	31.03.2027

1 Introduction

1.1 Setting

A university research chair, or a team of similar size, operates shared infrastructure: Linux hosts with graphics processors reached over Secure Shell (SSH), a Slack workspace, a GitHub organisation, and a handful of internal dashboards and databases. Its people change every semester, because thesis students, external collaborators, and working students join and leave on their own dates. Whoever needs access to any part of this infrastructure asks for it by chat message or e-mail.

1.2 Existing Infrastructure

Keycloak serves as the identity provider of the chair. It handles OpenID Connect (OIDC) login, groups, and profile attributes such as SSH public keys, and its Fine-Grained Admin Permissions restrict an administrative right to a single group [Key26]. The Linux hosts read this directory over the Lightweight Directory Access Protocol (LDAP) through `nslcd`, `sshd`, `pam_access`, and `sudoers`, and an Ansible role configures each host. Slack and GitHub manage membership through their own application programming interfaces (APIs)[1] and tokens, and neither shares an administrative interface with Keycloak.

2 Problem

Requesters do not know who manages which resource. A student who needs a host, a channel, and a repository asks several people and meanwhile borrows the session of a colleague or works under a shared account. Bauer et al. observed this workaround in far more mature organisations [Bau+09], and Beautement et al.

[1] https://api.slack.com/ and https://docs.github.com/en/rest

explain it as a limited compliance budget [BSW08]. The current process doesn't scale and is the worst bottleneck of the chair.

Maintainers, the staff who look after a project, know whether a request is legitimate but lack the rights to act on it, while the super admin holds the rights but lacks the context. The usual workaround hands the maintainer full administrative rights over every user and every host. It contradicts the protection principles of Saltzer and Schroeder [SS75] and leaves open the delegation problem of administrative role models [BS00, SBM99].

Access outlives its purpose and leaves no trace. No grant carries an expiry date, so access piles up and former students keep SSH access months after they leave, the same decay that Schaad et al. [SMJ01] and Kern et al. [Ker+02] report from industry. **Auditors** cannot reconstruct who held access when, or who approved it, because the history sits in scattered chat messages. A self-service platform that arms the workflow engine of Keycloak with an expiry date at grant time would end this accumulation.

3 Motivation

Access that ends on its own changes the question an organisation answers: it no longer asks who still needs a grant, but only who wants more time. The requester describes a need once, in terms of a project, and receives working access within minutes. The maintainer keeps the judgement over a request without holding any privilege in the identity provider, and the super admin works through a queue of vetted requests instead of interruptions. The auditor receives evidence as a by-product of normal operation.

Research on access control supplies the building blocks for this vision. Role-based access control gave organisations a shared vocabulary for who may do what [San+96], and Ferraiolo et al. consolidated its models into a reference standard [Fer+01]. Bertino et al. attach enabling and disabling events to roles in their

temporal model. [BBF01] Joshi et al. generalise this idea to role assignments with a limited duration [Jos+05].

4 Objective

I will build a platform on which any member of the chair requests time-limited access to any resource, a maintainer decides, and the access ends on its own. I follow the object-oriented software engineering method of Bruegge and Dutoit, and Figure 1 shows the actors and use cases of the platform as a use case diagram in the Unified Modeling Language. I pursue four objectives:

- Structure resources as an inheriting tree
- Design and implement connectors with expiry
- Delegate request review without privilege escalation
- Automate the recertification of grants

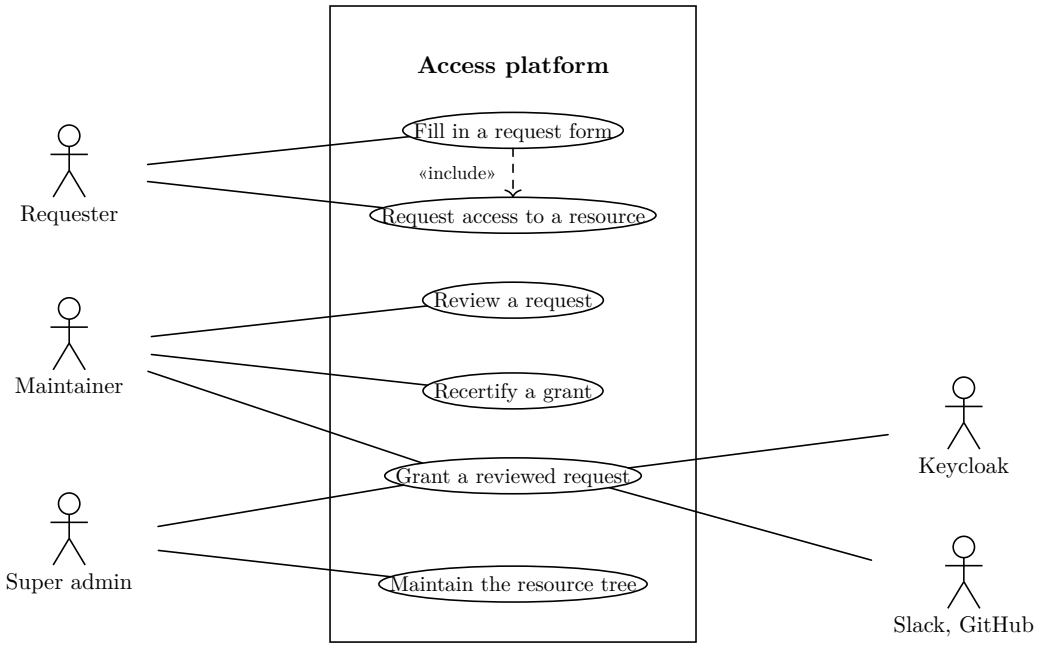


Figure 1: Use case diagram of the access platform. Requesters fill in forms and request access, maintainers review, grant, and recertify, and the super admin maintains the resource tree and completes every grant that needs privileged rights. Keycloak, Slack, and GitHub take part as secondary actors, because a grant takes effect only once their state changes.

4.1 Structure Resources as an Inheriting Tree

I will replace the flat list of hosts and channels with a resource tree whose inner nodes are folders and whose leaves each carry a binding to one target system. Maintainership, audit policy, and grants pass down from a folder to every resource beneath it. One approval on a project folder therefore covers its Slack channel and its GitHub team, and it covers a resource that a maintainer adds to the folder next month without a second decision.

Inheritance runs on every authorisation check, so it has to stay cheap. I will store the tree with a transitive-closure table of ancestor–descendant pairs and cap the fan-out, so that an approval on the root never reaches the whole infrastructure by accident. The objective is met when a grant on a folder with n bindable descendants yields exactly n provisioning commands and a resource added later inherits the grant without a new request.

4.2 Design and Implement Connectors with Expiry

A Slack channel, a GitHub team, and a Keycloak group share only the idea of membership. I will define one connector contract that validates a binding, grants and revokes access, and optionally lists the current members, and I will implement the `keycloak-group`, `slack-channel`, `github-team`, and `webhook` connectors on it. The super admin configures their credentials at runtime, and Figure 2 shows how the components of the platform connect.

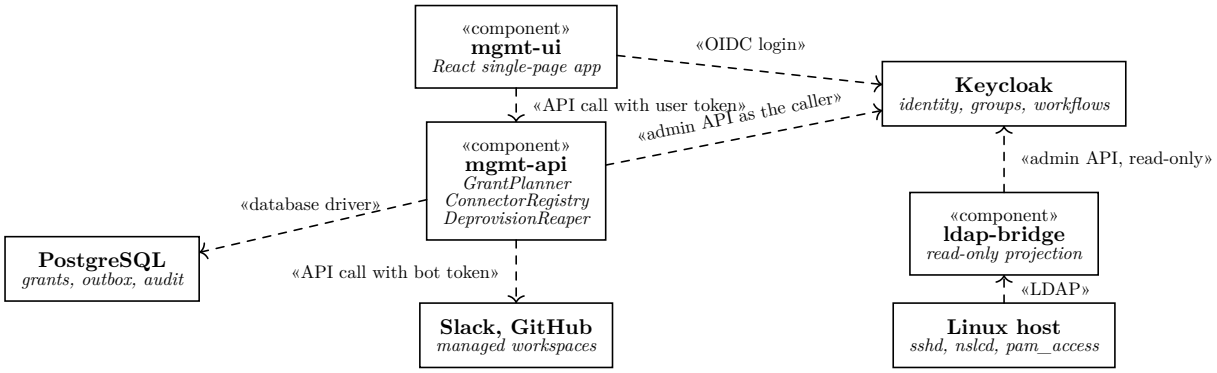


Figure 2: System Architecture

The platform changes Keycloak only with the token of the calling user, so it arms the workflow engine of Keycloak at grant time. A deprovisioning reaper revokes the grants of the other connectors through an outbox, and a drift check reports memberships that outlive their grants. An automated scenario checks that SSH, Slack, and GitHub access from one approval disappears after expiry, even when the workflow engine fails. I measure extensibility by the size of the change that the `webhook` connector needs after I freeze the framework.

4.3 Delegate Request Review Without Privilege Escalation

A requester should not need to know which resources a project needs. I will build declarative request forms whose fields, visibility conditions, and rules turn answers into requests for a resource path, a level, and a duration. The server evaluates the answers itself, so a submission cannot request a resource that the form does not offer. One onboarding submission thereby replaces several conversations.

Keycloak requires a realm-wide administrative role for its workflow API, and no resource owner may hold that role. I will therefore split the approval: in the review step, the resource owner records a decision without changing Keycloak, and in the grant step, the super admin sets membership and expiry atomically, in bulk for a vetted queue. The resource owner completes a request alone when it touches only non-privileged bindings, such as a Slack channel. I will compare the manual administrative actions and the time to working access with the current process.

4.4 Automate the Recertification of Grants

I will attach a recertification interval to each resource, which its descendants inherit, and open a campaign whenever the interval elapses. The maintainer decides to keep or revoke each grant, and the platform stores every decision as an immutable record. Ignored campaigns escalate and may revoke grants automatically, but never in a campaign that reached nobody. The platform logs every

change with the person who made it, and the admin event log of Keycloak keeps a second copy.

5 Schedule

The thesis runs from 1 October 2026 to 31 March 2027, and I divide it into iterations of two to four weeks. Each iteration ends with an increment that a user can try in the web interface, and each item names the objective it serves and the outcome that I demonstrate. Later iterations extend the platform that the earlier iterations deliver.

- **Iteration 1 (1–28 October), Objective 1:** Keycloak realm, OIDC login, SSH key self-service, and resource tree. Outcome: a super admin delegates a folder to a maintainer.
- **Iteration 2 (29 October – 25 November), Objectives 2 and 3:** requests, review and grant steps, `keycloak-group` connector, LDAP bridge, and Ansible role. Outcome: an SSH login ends at expiry.
- **Iteration 3 (26 November – 23 December), Objective 2:** connector framework with outbox and reaper, Slack and GitHub connectors. Outcome: one approval grants and revokes both.
- **Iteration 4 (24 December – 20 January), Objective 3:** request forms with builder and server-side evaluation. Outcome: one submission yields requests for several connectors.
- **Iteration 5 (21 January – 17 February), Objective 4:** recertification campaigns, drift check, and e-mail notifications. Outcome: an ignored campaign escalates on schedule.
- **Iteration 6 (18 February – 17 March), Objectives 2 and 3:** `webhook` connector and deployment at the chair. Outcome: the correctness scenarios pass, and I measure the effort.
- **Iteration 7 (18–31 March):** Write the thesis and prepare the final presentation.

Bibliography

[Key26] Keycloak Authors, "Keycloak Server Administration Guide." Accessed: Oct. 18, 2026. [Online]. Available: https://www.keycloak.org/docs/latest/server_admin/

[Bau+09] L. Bauer, L. F. Cranor, R. W. Reeder, M. K. Reiter, and K. Vaniea, "Real Life Challenges in Access-Control Management," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, ACM, 2009, pp. 899–908. doi: 10.1145/1518701.1518838.

[BSW08] A. Beautement, M. A. Sasse, and M. Wonham, "The Compliance Budget: Managing Security Behaviour in Organisations," in *Proceedings of the 2008 New Security Paradigms Workshop (NSPW)*, ACM, 2008, pp. 47–58. doi: 10.1145/1595676.1595684.

[SS75] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.

[SBM99] R. Sandhu, V. Bhamidipati, and Q. Munawer, "The ARBAC97 Model for Role-Based Administration of Roles," *ACM Transactions on Information and System Security*, vol. 2, no. 1, pp. 105–135, 1999, doi: 10.1145/300830.300839.

[BS00] E. S. Barka and R. S. Sandhu, "Framework for Role-Based Delegation Models," in *Proceedings of the 16th Annual Computer Security Applications Conference (ACSAC)*, IEEE, 2000. doi: 10.1109/ACSAC.2000.898870.

[SMJ01] A. Schaad, J. Moffett, and J. Jacob, "The Role-Based Access Control System of a European Bank: A Case Study and Discussion," in *Proceedings of the 6th ACM Symposium on Access Control Models and Technologies (SACMAT)*, ACM, 2001, pp. 3–9. doi: 10.1145/373256.373257.

[Ker+02] A. Kern, M. Kuhlmann, A. Schaad, and J. Moffett, "Observations on the Role Life-Cycle in the Context of Enterprise Security Management," in *Proceedings of the 7th ACM Symposium on Access Control Models and Technologies (SACMAT)*, ACM, 2002, pp. 43–51. doi: 10.1145/507711.507718.

[San+96] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845.

[Fer+01] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001, doi: 10.1145/501978.501980.

[BBF01] E. Bertino, P. A. Bonatti, and E. Ferrari, "TRBAC: A Temporal Role-Based Access Control Model," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 191–233, 2001, doi: 10.1145/501978.501979.

[Jos+05] J. B. D. Joshi, E. Bertino, U. Latif, and A. Ghafoor, "A Generalized Temporal Role-Based Access Control Model," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 1, pp. 4–23, 2005, doi: 10.1109/TKDE.2005.1.

[UD26] S. Uçan and A. Demirel Uçan, "Generative AI in Student Writing: A Six–Level Taxonomy for Responsible Use in Higher Education Assessment," *Journal of Academic Ethics*, vol. 24, no. 2, p. 63, 2026, doi: 10.1007/s10805-026-09739-y.

Transparency in the Use of AI Tools

I used the following generative artificial intelligence (AI) tool while preparing this proposal. The level refers to the six-level taxonomy of generative AI use in writing by Uçan and Demirel Uçan [UD26].

Tool	**Model / Version**	**Date(s)**	**Level**	**Purpose**	**Scope**
Anthropic Claude Code	Claude Opus 5.5 (claude-opus-5-5)	18.10.2026	5	Drafting text, authoring the Typst sources and diagrams, researching references	Sections 1-5, Figures 1-2, bibliography entries